

DCLT Portal — Conceptual Architecture

Purpose

This is the canonical conceptual reference for the DCLT Portal. It defines the data model, the disciplines that govern it, and the vocabulary used to describe it. Two audiences:

- **Project conversations.** This document primes work in this project. New threads inherit its model without re-deriving it.
- **Claude Code.** This document is the upstream authority for any Technical Architecture, schema, or implementation work. Claude Code consumes it and produces the Technical Architecture (TA), which extends but never overrides what's here.

When implementation surfaces gaps or contradictions, the resolution path is to update this document first, then propagate to the TA. The TA never silently diverges.

Scope

This document defines:

- The Tag / Layer dichotomy and the authorship test that separates them
- Layer source types (External / Derived / Dynamic) and their refresh disciplines
- Tag dimension structure (state space, default, applicability, transitions)
- Composition rules between Tags and Layers, and cycle prevention
- Filtering semantics, including applicability and the Not Applicable affordance
- Naming canon
- Worked examples grounding the abstractions

This document does not define:

- Schema DDL, column types, indexes, triggers
- File layouts, module boundaries, package structure
- Ingest pipeline implementation, OCR processing, deployment
- Derived Layer materialization order or scheduling

- Dynamic Layer evaluation mechanism or caching
- Query patterns, performance optimization, escalation thresholds
- UI surface implementation
- API shape, authentication, deployment

Those belong in the Technical Architecture.

Guiding principles

The model exists to support four commitments:

Hard data is gettable. What's been ingested was hard-won; what's asked for next is probably feasible. The architecture should encourage incremental sourcing rather than presuming everything is known up front.

The interface serves the question. Same data, different lenses, depending on what's being decided. Filtering and presentation are first-class concerns, not an afterthought on top of storage.

Human judgment is captured, not lost. Tags, notes, and determinations from people who know the ground are first-class data, distinct from machine guesses, with full provenance.

Lineage is non-negotiable. Every answer traces to its source. The split between machine-extracted and human-confirmed is preserved in every read.

These principles drive specific design choices: append-only Tags so judgment survives re-ingest; the Layer/Tag separation so machine and human authorship don't collide; applicability as a query concern so dimensions can be conditional without forcing dummy states; node-typed Layers so polymorphism doesn't leak into consumers.

The loop

The architecture exists to support a four-step cycle:

1. **Surface what's there.** External Layers source from feeds; Derived Layers calculate from those.
2. **Ask what we could do with it if accurate.** Dynamic Layers and queries pose the question.
3. **Let people who know the ground convert guesses into determinations.** Tag dimensions are the workflow.

4. **Re-ask with stronger footing.** Each pass, the latest fold answers with more credibility.

Re-ingest never disturbs human work — it lives in Tags. Changing a Layer formula never disturbs history — Layers don't have history. Changing a calculation never disturbs storage — Derived Layers re-materialize on next ingest, Dynamic Layers re-evaluate on next read, neither touches `transactions.db`.

Where this lives today

Single-server portal, VPS-hosted, GitHub-deployed. Local processing pipeline produces a read-only `reference.db` containing External and (after refactor) Derived Layers, deployed alongside the application. `transactions.db` lives on the server, append-only, never overwritten by deployments. Three-pane parcel view is the primary UI surface; document-centric views and cover-page hygiene rollups are near-term.

The dividing line

Authorship in the portal is the only test.

Attribute. Not authored by users in the portal. Sourced from external feeds, derived by calculation from other Layers, or computed dynamically using Tag inputs. Lives in a Layer.

Tag. Authored by users in the portal. Default-seeded so the dimension is queryable from day one, then assorted by users as work progresses. Lives in `transactions.db`.

The seam is authorship of the *current value*, not authorship of the *seed*. A Tag whose default is a system suggestion is still a Tag — the seed is a proposal, the user's assortment is the verdict, and human authorship beats the seed regardless of timing. An Attribute whose calculation depends on user-authored Tags is still an Attribute — no user assorts the calculated value directly.

Layers (attributes)

Every Attribute belongs to a Layer. A Layer has two classifying axes.

Source type

Three values.

- **External.** Sourced from a feed. Assessor exports, MassGIS layers, Registry OCR output, Dave's ownership spreadsheet.
- **Derived.** Calculated from other Layers (External or Derived only). Materialized in

`reference.db` . Refreshed on ingest.

- **Dynamic.** Calculated using at least one Tag fold as input. Never materialized. Evaluated at read time.

A Layer is Dynamic if it reads any Tag fold, Derived otherwise. Classification follows the inputs, not the intent. Adding a Tag input to a Derived Layer reclassifies it — code review catches the change.

Node type

The output key. Parcel, Document, or any future node type. A Layer that aggregates document data into per-parcel rollups is a parcel-typed Layer with document inputs — node type follows the output, not the inputs.

Naming

`<NodeType><Subject>` for derived and dynamic Layers, e.g. `ParcelFootprintRatio` , `DocumentKeywordScores` , `ParcelCoverageRollup` . External Layers may inherit names from their source schema.

Storage and refresh

Source type	Storage	Refresh
External	<code>reference.db</code>	Ingest
Derived	<code>reference.db</code> (materialized)	Ingest
Dynamic	In memory (formula only)	Per read

If a Dynamic Layer becomes a performance problem, the escalation is an overnight materialization job — caching, not a storage commitment. The Layer’s classification doesn’t change; only its read path does.

Tags

A Tag is a dimension for assorting nodes (parcels, documents, others) with a finite state space. Users move records through that space. The current state on a dimension is the latest-fold of the tag log for that `(target_type, target_id, dimension)` .

Mechanics

Append-only log in `transactions.db`. Every event records target (`target_type` , `target_id`), dimension, state, `user_id` , server-assigned monotonic `seq` , client `created_at` , optional note. No UPDATE, no DELETE — corrections append a new event. Latest row wins per (`target_type` , `target_id` , `dimension`) . Full history preserved and queryable.

A database-level trigger rejects UPDATE and DELETE on the tag table. Application code paths must never construct such statements.

Dimension definition

Each Tag dimension is defined by six properties, all in code:

1. **Name.** `CoverageDetermination` , `IdentityResolution` , `Article97Determination` , `DCLTOwnership` .
2. **Node type.** Which target type the dimension applies to.
3. **State space.** Closed set of valid states. Includes `Unconfirmed` if the dimension is a decision-pending workflow. Does not include `Not Applicable` — applicability is a separate axis (see Filtering).
4. **Default rule.** How new records seed. Constant (usually `Unconfirmed`), or a function of attributes. Default rules read External or Derived Layers only — never Dynamic — to keep the dependency graph acyclic.
5. **Applicability rule.** Which records the dimension is meaningful for. May depend on attributes. `IdentityResolution` is dormant when `IdentityState = OK` . `Article97Determination` only applies to documents with a keyword hit above threshold.
6. **Allowed transitions.** Usually any-to-any. Sometimes constrained by an attribute: `IdentityResolution` permits {ADB Add, GIS Remove} when `IdentityState = GIS-only` , {GIS Add, ADB Remove} when `ADB-only` .

Adding a Tag dimension is a small commit, not a runtime config change. Dimensions are versioned with the code; the log preserves history through definition changes.

Hygiene journey

`Unconfirmed` is the work queue. Filter to it, process, watch coverage Layers climb. Derived or Dynamic Layers like `% adjudicated` and `% resolved` make progress visible across the inventory.

Naming convention

- `-Determination` for categorical verdicts where users decide what an observation means.
- `-Resolution` for dispositions on a discrepancy.
- Plain name where the dimension itself is a workflow with no parallel attribute (`DCLTOwnership` when the source is users-marking-in-the-portal).

One suffix per Tag shape keeps the model legible as the dimension count grows.

Composition

Layers and Tags compose by reading from each other under constrained rules.

Layers can read Tags. A Layer that reads any Tag fold is Dynamic. This is the only way a Tag's current state influences a calculated value.

Tag default and applicability rules can read Attributes. Default seeds and applicability conditions may depend on External or Derived Layers. They may not depend on Dynamic Layers — that would create a cycle (Tag → Dynamic Layer → Tag).

Tags do not read Tags. A Tag's state space and seed are independent of other Tags' folds. If a workflow seems to require otherwise, the right shape is a Dynamic Layer that combines the Tags, queried wherever the combination matters.

Attributes and Tags never write to the same row. They live in different stores with different keys. The "human beats system" fold rule is automatic, not enforced — there is nothing for system writes to overwrite.

Filtering and applicability

Filtering is a first-class concern. The model treats applicability as a separate axis from state, not a state value.

Multi-state pickers

When a user filters on a Tag, the UI offers a multi-picker over the dimension's state space. A record matches if its current fold is among the selected states *and* the dimension is applicable to it. If the dimension's applicability rule is conditional, the picker also offers `Not Applicable` as a structurally separate affordance — visually distinct from state choices, since it filters on a different axis.

For dimensions whose applicability rule is "all records," `Not Applicable` is omitted (the result would always be empty). For dimensions like `IdentityResolution` or

`Article97Determination` , `Not Applicable` is present and meaningful.

Filter semantics

- Selecting state `s` matches records where `applicable AND fold = s` .
- Selecting `Not Applicable` matches records where `NOT applicable` , regardless of fold.
- Selecting both states and `Not Applicable` returns the union.

Rollup denominators

Derived and Dynamic Layers that report dimension-level percentages count among the *applicable* population by default. `ParcelArticle97Rollup` 's "% confirmed" reads against documents with keyword hits, not against all documents — otherwise the hygiene number is structurally diluted. The `Not Applicable` count is reported separately when relevant, never folded into the active denominator.

Applicability flips

Applicability is computed at read time from current attribute values. A record can transition between applicable and not-applicable as upstream data changes. When that happens, the existing Tag fold is preserved in history and remains queryable, but the record falls under `Not Applicable` for active filters until applicability is restored.

Example: a parcel was `GIS-only` , the user resolved it as `ADB Add` , and the next ingest updates the Assessor's database so `IdentityState` flips to `OK` . The fold remains `ADB Add` in history; the active filter shows the parcel as `Not Applicable` . The resolution did its job and the dimension stepped back.

Examples

Coverage

`ParcelFootprintRatio` is a Derived Layer (footprint area ÷ parcel area, computed from External Layers).

`CoverageDetermination` is a Tag.

- **States:** {`Unconfirmed`, `Undeveloped`, `Underdeveloped`, `Developed`}
- **Node type:** parcel
- **Default:** `Unconfirmed`

- Applicability: all parcels
- Transitions: any-to-any

`ParcelCoverageRollup` is a Dynamic Layer producing inventory-level percentages by determination state — drives the cover-page hygiene journey.

Identity

`IdentityState` is a Derived Layer with states `{OK, ADB-only, GIS-only}`, calculated each ingest from External Layers (Assessor and GIS presence).

`IdentityResolution` is a Tag.

- **States:** `{Unconfirmed, ADB Add, ADB Remove, GIS Add, GIS Remove}`
- Node type: parcel
- Default: `Unconfirmed`
- Applicability: only when `IdentityState ≠ OK`
- Transitions: constrained by `IdentityState` (`GIS-only → {ADB Add, GIS Remove}`; `ADB-only → {GIS Add, ADB Remove}`)

Article 97

`DocumentArticle97KeywordScore` is an External Layer (OCR keyword scorer output).

`Article97Determination` is a Tag.

- **States:** `{Unconfirmed, Confirmed, Denied}`
- Node type: document
- Default: `Unconfirmed` when keyword score is above threshold; dimension inapplicable otherwise
- Applicability: documents with keyword score above threshold
- Transitions: any-to-any among the three states

`ParcelArticle97Rollup` is a Dynamic Layer aggregating document determinations to the parcel level (count confirmed, count unconfirmed, etc.). Denominator is applicable documents.

DCLT ownership

Two cases, depending on the source of truth.

If users mark ownership directly in the portal: `DCLTOwnership` is a Tag with states `{Unconfirmed, Owned, Not owned}`, applicability all parcels.

If Dave's spreadsheet is the source of truth and arrives by feed: `ParcelDCLTOwnership` is an External Layer with states `{owned, not listed}` and an as-of date. A Tag may still exist on top to record portal-side overrides when a user notices the spreadsheet is wrong — same shape as `Article97` (attribute observes, tag overrides).

Migration notes

The current implementation is expected to have:

- A single tags surface mixing system-written and user-written rows.
- Dimensions whose seeds are written by ingest and treated as authoritative until overwritten by a user.
- Calculated values co-mingled with stored ones, distinguished informally if at all.

The refactor:

1. **Inventory existing tag dimensions.** Classify each as Tag (user-authored in portal) or Attribute (system-authored). The "system tag" category disappears.
2. **Move attribute dimensions into Layers.** Decide source type (External / Derived / Dynamic) per dimension based on inputs.
3. **Restrict the Tags table to portal-authored events only.** Add the UPDATE/DELETE trigger if not present. Audit existing rows; system-written rows migrate to their Layer's storage.
4. **Rebuild seeds-from-system as default rules on Tag dimensions.** The default reads the relevant Layer at fold time rather than being written into the log at ingest.
5. **Audit read paths.** Every place that reads a "tag" today, decide whether the consumer wants the Tag fold (decision state) or the Layer value (observation). Update accordingly.
6. **Audit filtering UIs.** Replace any filter that treats applicability as a state value with the two-axis pattern (state multi-picker + separate Not Applicable affordance).

The `reference.db / transactions.db` split survives unchanged. What changes is what goes on each side of the line.

Naming canon

Claude Code and downstream documents should use these terms verbatim. Synonyms and parallel vocabulary fragment the model.

Categories: Tag, Layer, Attribute, Dimension. **Layer source types:** External, Derived, Dynamic. **Node types:** Parcel, Document. Future node types added by code change, not configuration. **Tag mechanics:** fold (verb and noun), latest-wins, append-only, target (`target_type`, `target_id`), `seq`, applicability, applicable, Not Applicable. **Dimension parts:** state space, default rule, applicability rule, allowed transitions, transition. **Tag suffixes:** `-Determination`, `-Resolution`, plain workflow name. **Stores:** `reference.db`, `transactions.db`. **Pipeline:** ingest, materialize, evaluate, refresh.

Banned vocabulary:

- “system tag,” “user tag,” “automatic tag” — the old conflation. Use Tag (always user-authored) or Attribute (anything system-sourced or calculated).
- “calculated attribute” as a stored on-disk artifact other than a materialized Derived Layer. If it reads Tags, it’s Dynamic and lives only as a formula.
- “update tag,” “delete tag” — Tags are append-only. Corrections are new events.
- “Not Applicable” as a state in any state space. It’s a query-time projection.

How the Technical Architecture extends this

The TA is the implementation companion to this document. It is downstream — it implements what’s defined here, in the vocabulary defined here.

The TA covers:

- Schema DDL: tables, columns, types, primary and foreign keys, the WORM trigger
- Indexes supporting the latest-wins fold and history retrieval
- File and module organization
- Ingest pipeline: fetching, normalization, joining against the parcel spine, OCR processing
- Derived Layer materialization: order, dependency resolution, rebuild semantics
- Dynamic Layer evaluation: where formulas live, how they’re invoked, caching strategy
- Tag write path: validation against state space, applicability check, transition check
- Query patterns combining attribute filters, tag fold filters, and applicability projections

- Performance: where caching applies, escalation criteria for materializing a Dynamic Layer overnight
- Migration mechanics for moving from the current implementation
- UI surface implementation, including the two-axis filter pattern
- API shape, deployment, environment

The TA must not:

- Redefine, rename, or split the categories defined here (Tag, Layer, Attribute, Dimension)
- Introduce parallel vocabulary or new top-level abstractions without first proposing them as a CA change
- Decide composition rules that contradict the rules in this document
- Add states like “Not Applicable” to a dimension’s state space
- Materialize Dynamic Layers as a default storage strategy (caching is acceptable; storage commitment is not)
- Allow UPDATE or DELETE on Tags under any circumstance

When the TA needs more than this document provides:

The TA may surface gaps: examples that don’t fit cleanly, composition patterns this document didn’t anticipate, applicability or transition rules that need a position. The resolution path is to propose an update to this CA, get it accepted, and then update the TA. The TA does not extend the model unilaterally.

Evolution

This document is canonical for the model but not frozen. Implementation will surface things the model didn’t anticipate. When that happens:

1. The gap or contradiction is named explicitly.
2. A position is taken in this document — new section, amended rule, additional example.
3. The TA is updated to match.

This document supersedes the prior `dcltportalsummary.pdf` and `DCLT_Tags_Layers_Architecture.pdf`. Future revisions are tracked in version control alongside the codebase.

